

Chapter 5: Artificial Neural Network

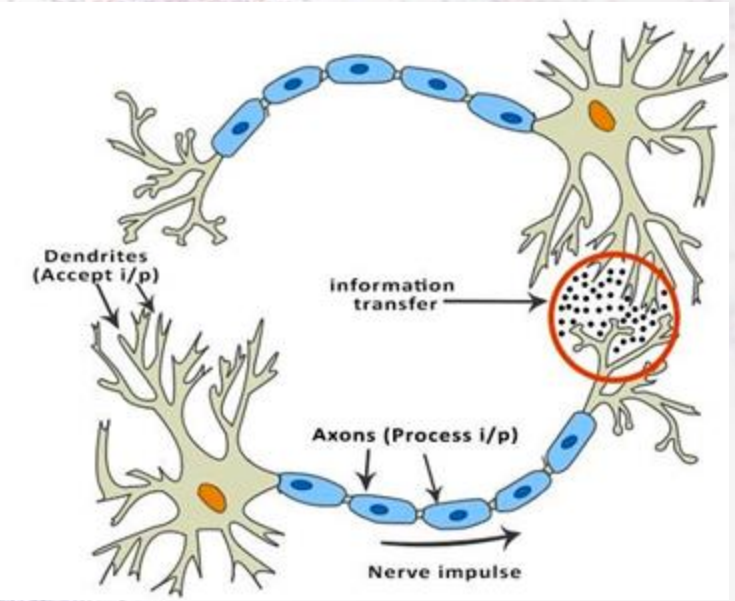
Ph.D. Nguyen Thi Khanh Tien

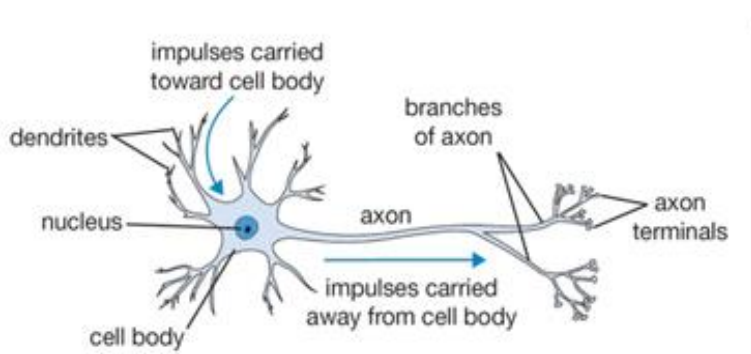
tiennttk@ut.edu.vn

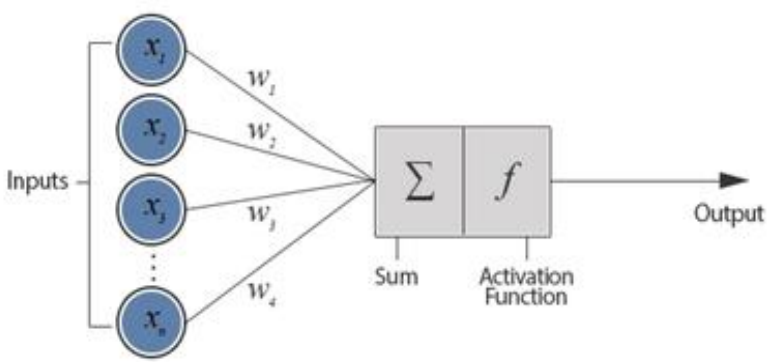
Introduction to Artificial Neural Networks (ANNs)

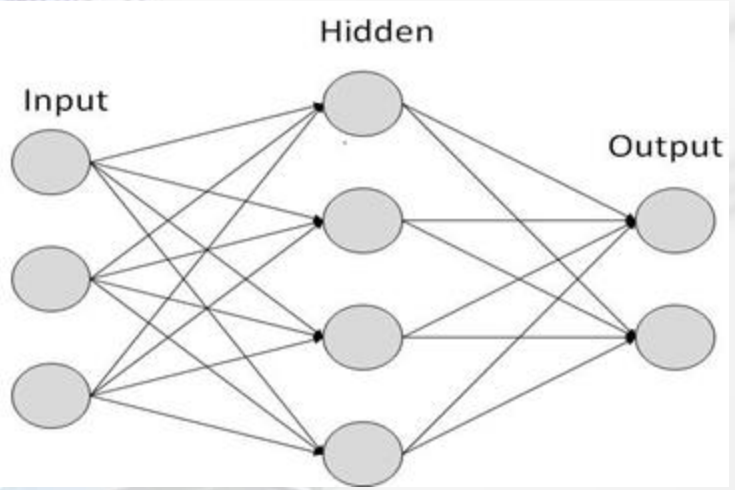
Artificial Neural Networks (ANNs) are inspired by the human brain, which consists of billions of interconnected neurons, and aim to mimic their learning and decision-making processes using artificial neurons and connections.

- ANNs are composed of multiple ***nodes***, which imitate biological neurons of human brain.
- The output at each node is called its ***activation*** or ***node value***.
- Each link is associated with ***weight***. ANNs are capable of learning, which takes place by altering weight values.









A Brief History of Neural Nets

Increase of data transmission speed

1940 ~ 1970: The 1st AI Boom | 1980 ~ 1990: The 2nd Boom | 2006 ~: The 3rd Boom

The advent of the idea of AI



1958: Perceptron

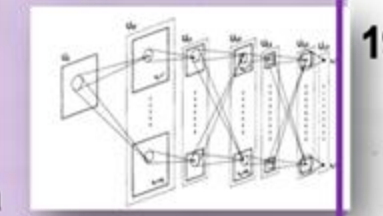
1968: "2001 : A Space Odyssey"



1st AI Winter

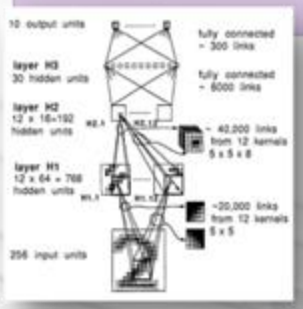
- Limitation of hardware
- Lack of computation algorithms
- Hardships in linearly inseparable data problems

1980: Neocognitron



1986: Backpropagation

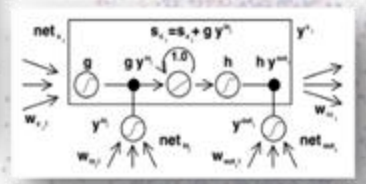
1989: The first practical CNN



2001: Release of Xbox



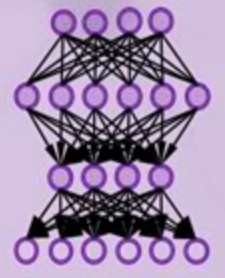
1997: LSTM



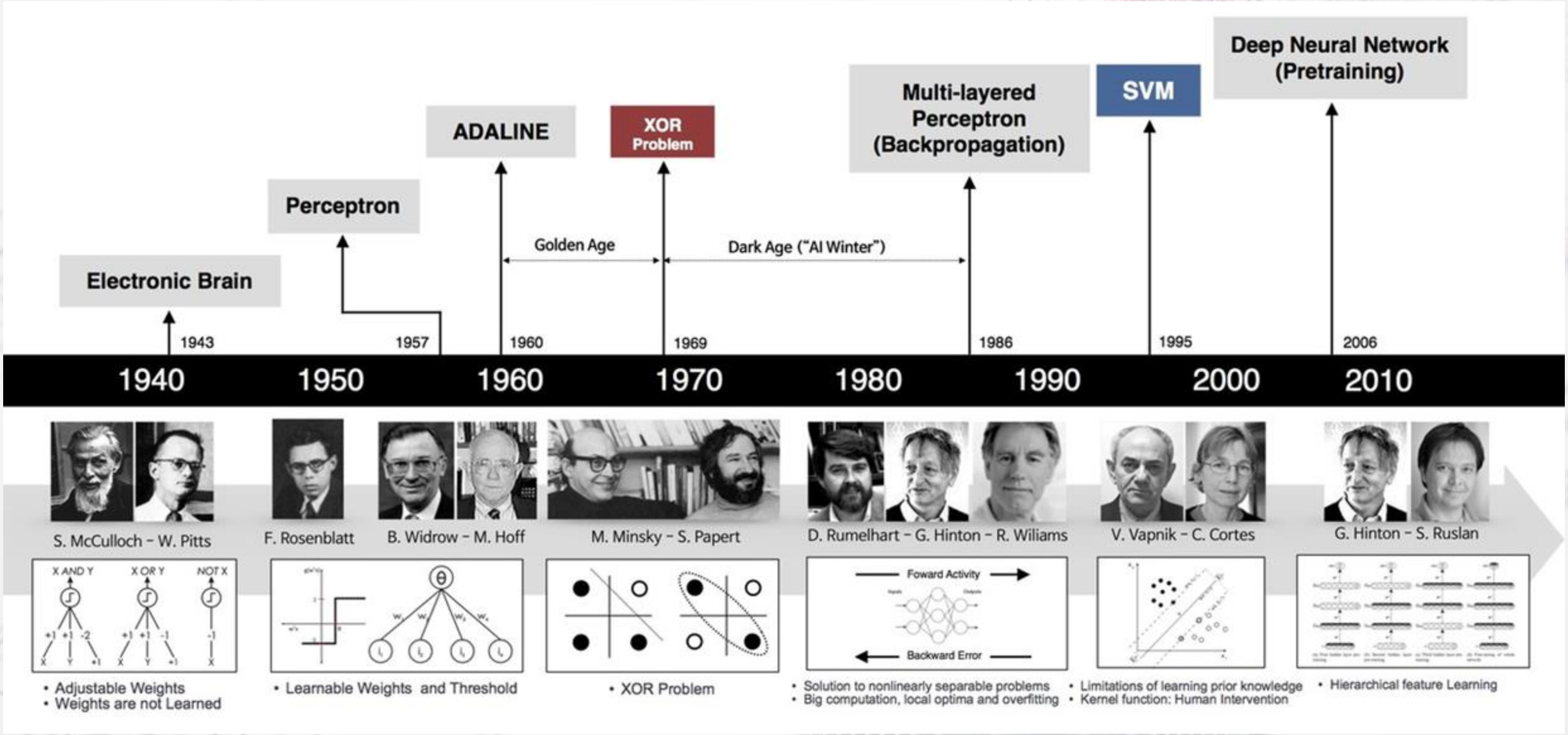
2nd AI Winter

- Limitation of hardware
- Shortage of data sources
- Lack of theories for hyper parameters
- <u>Vanishing / exploding gradient problem</u>

2006: Pretraining of deep belief net



History of neural networks



History of neural networks

1943: Warren S. McCulloch and Walter Pitts published "A logical calculus of the ideas immanent in nervous activity (link resides outside ibm.com)" This research sought to understand how the human brain could produce complex patterns through connected brain cells, or neurons. One of the main ideas that came out of this work was the comparison of neurons with a binary threshold to Boolean logic (i.e., 0/1 or true/false statements).

1958: Frank Rosenblatt is credited with the development of the perceptron, documented in his research, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain" (link resides outside ibm.com). He takes McCulloch and Pitt's work a step further by introducing weights to the equation. Leveraging an IBM 704, Rosenblatt was able to get a computer to learn how to distinguish cards marked on the left vs. cards marked on the right.

1974: While numerous researchers contributed to the idea of backpropagation, Paul Werbos was the first person in the US to note its application within neural networks within his PhD thesis (link resides outside ibm.com).

1989: Yann LeCun published a paper (link resides outside ibm.com) illustrating how the use of constraints in backpropagation and its integration into the neural network architecture can be used to train algorithms. This research successfully leveraged a neural network to recognize hand-written zip code digits provided by the U.S. Postal Service.

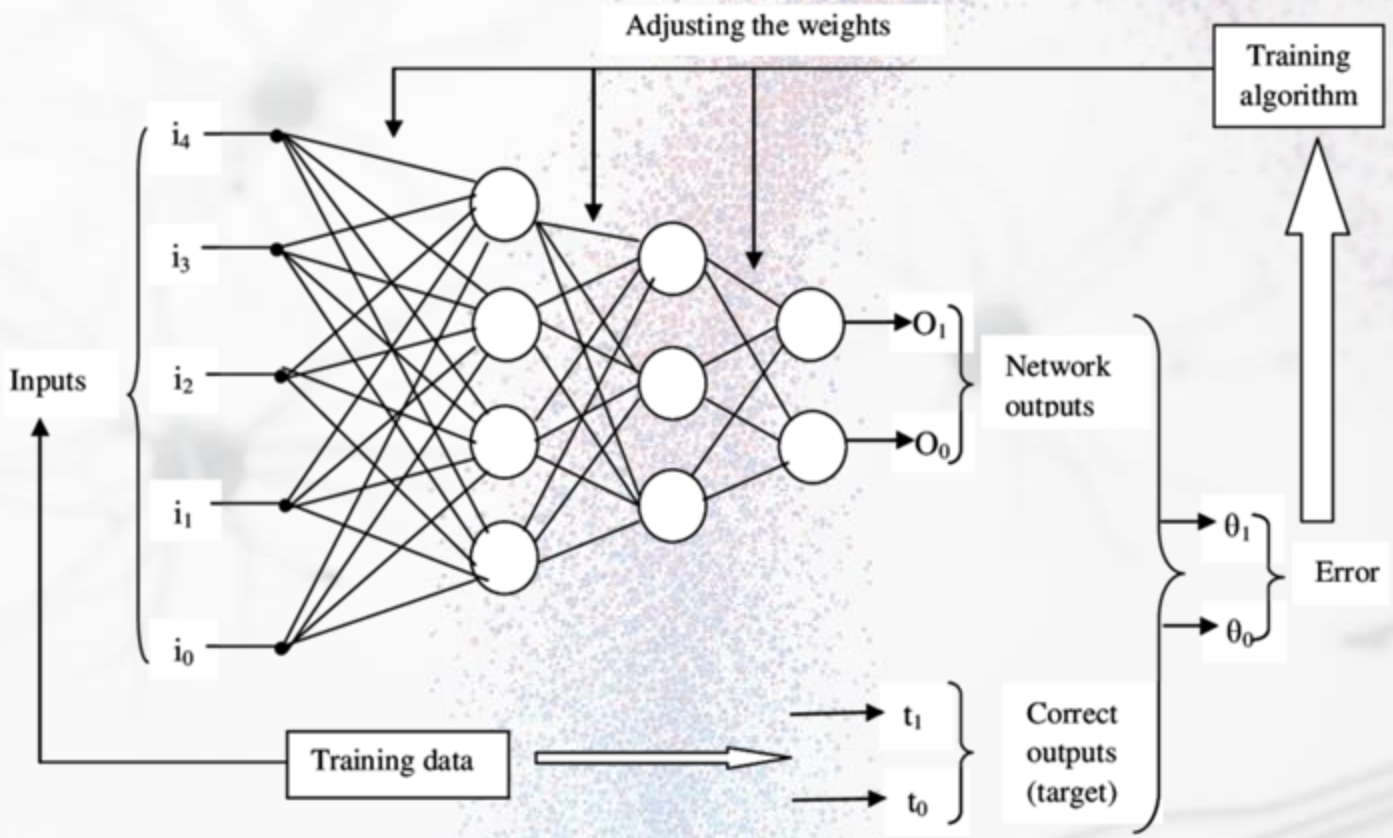
How do neural networks work?

The nodes process input data through mathematical calculations, similar to how biological neurons transmit information.

Here's how it works:

1. **Input Layer:** initial data for the neural network. Received raw data - images or text.
2. **Hidden Layers:** These layers process the input data, extracting features and patterns.
3. **Output Layer:** Produces the final result, like a classification or prediction.

During training, the network adjusts the connections between nodes (weights) to minimize errors in its predictions. This process, called backpropagation, allows the network to learn and improve its performance over time.



Each individual node as its own linear regression model, composed of input data, weights, a bias (or threshold), and an output.

$$\sum wixi + bias = w1x1 + w2x2 + w3x3 + bias$$

$$output = f(x) = 1 \text{ if } \sum w1x1 + b >= 0; 0 \text{ if } \sum w1x1 + b < 0$$

Back Propagation Algorithm

Backpropagation is a fundamental algorithm used to train neural networks. It's a supervised learning technique that adjusts the weights and biases of the network to minimize the difference between the predicted output and the actual target output.

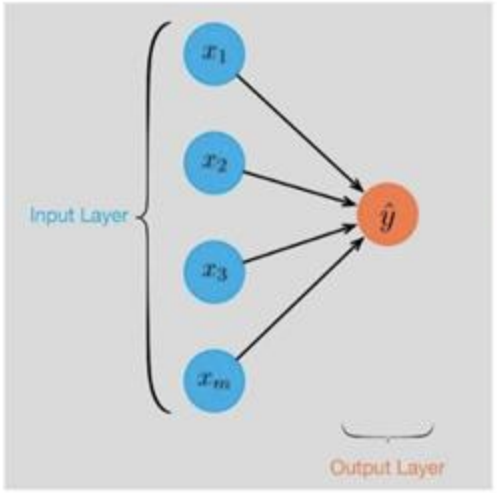
Here's a simplified breakdown of how it works:

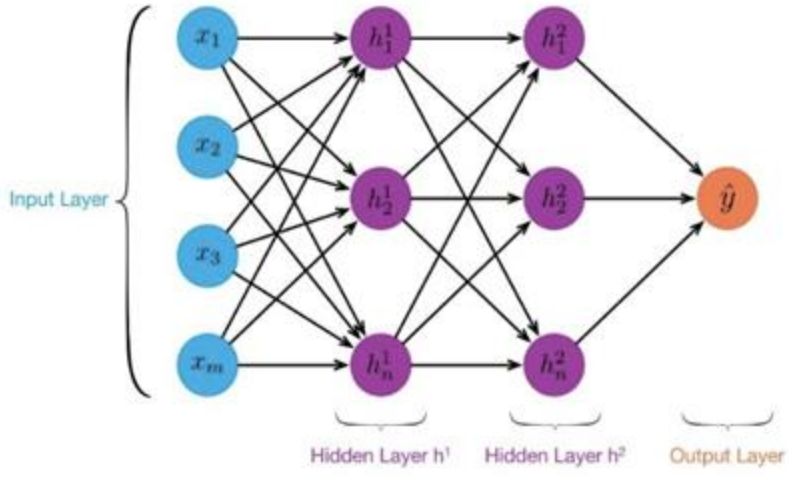
1. **Forward Pass:**
    - Input data is fed into the neural network.
    - The data is processed through each layer, with each neuron performing a weighted sum of its inputs and applying an activation function.
    - The output of the final layer is compared to the desired output, and an error is calculated.
2. **Backward Pass:**
    - The error is propagated backward through the network, layer by layer.
    - At each layer, the error is used to calculate the gradient of the error with respect to the weights and biases of that layer.
    - These gradients indicate the direction and magnitude of the weight and bias adjustments needed to reduce the error.
3. **Weight and Bias Update:**
    - The weights and biases are updated using an optimization algorithm like gradient descent.
    - This process is repeated for multiple iterations, gradually reducing the error and improving the network's accuracy.

Key Concepts:

- **Gradient Descent:** An optimization algorithm that iteratively adjusts parameters to minimize a cost function.
- **Learning Rate:** A parameter that controls the step size during gradient descent.
- **Activation Function:** A function that introduces non-linearity into the network, allowing it to learn complex patterns.
- **Loss Function:** A measure of how well the network is performing, used to calculate the error.

Forward propagation (Forward pass)





First, if we have ***m*** input data (***x1, x2, …, xm***), we call this ***m features***. A feature is just one variable we consider as having an influence to a specific outcome.

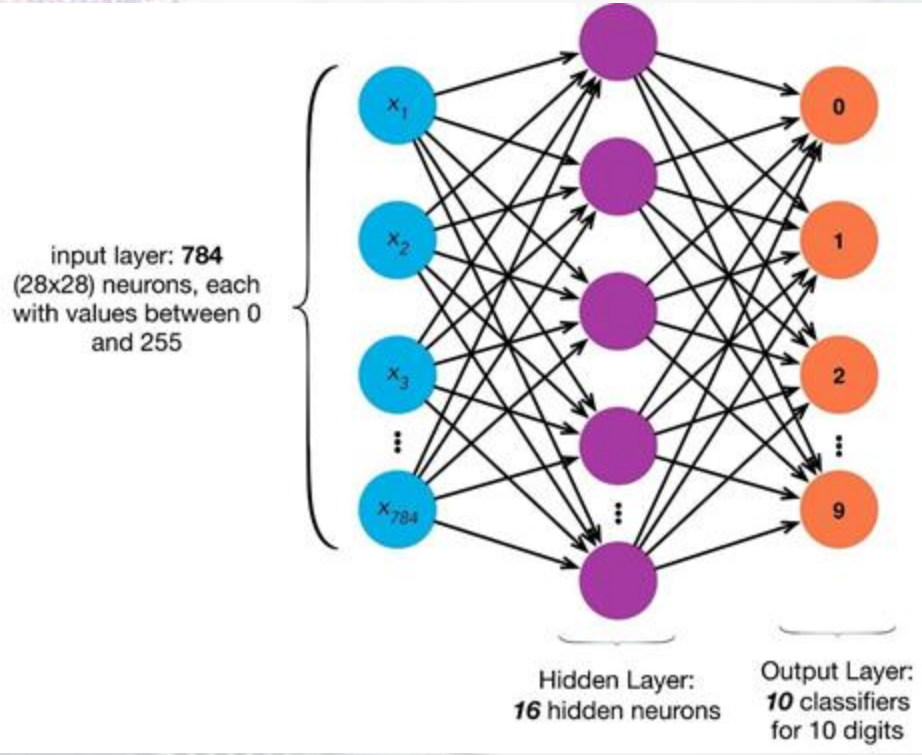
Secondly, when we multiply each of the m features with a weight (***w1, w2, …, wm***) and sum them all together, this is a **dot product**:

$$W \cdot X = w_1 x_1 + w_2 x_2 + \ldots + w_m x_m = \sum_{i=1}^{m} w_i x_i$$

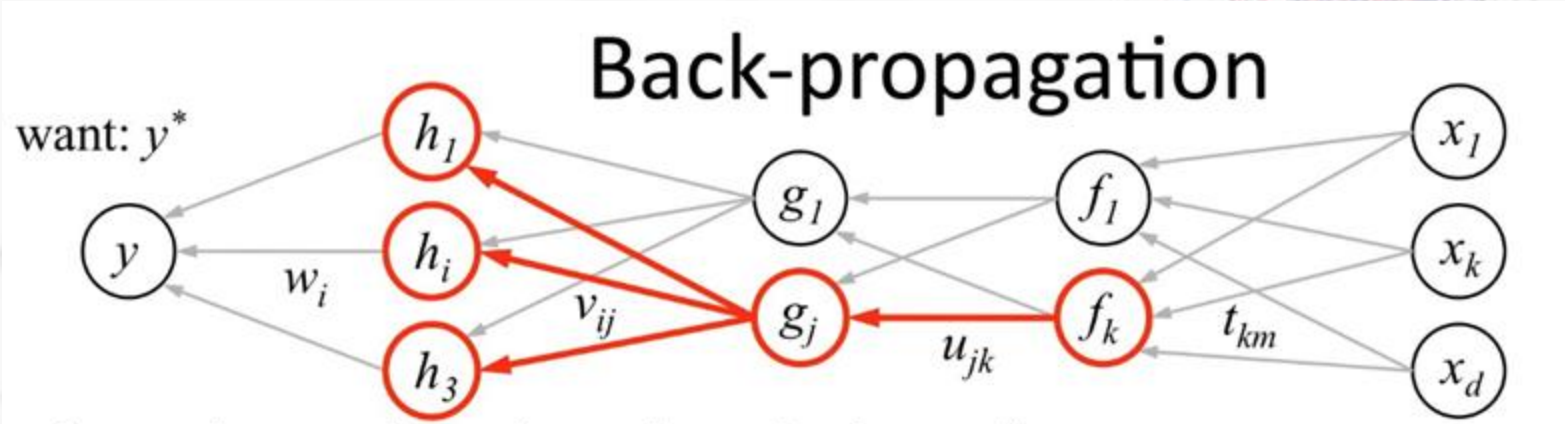
So here are the takeaways for now:

1. With ***m features*** in input ***X,*** you need ***m*** weights to perform a dot product
2. With ***n*** hidden neurons in the hidden layer, you need ***n*** sets of weights (***W1, W2, … Wn***) for performing dot products
3. With 1 hidden layer, you perform ***n*** dot products to get the hidden output ***h:*** (***h1, h2, …, hn***)
4. Then it's just like a single-layer perceptron, we use hidden output ***h:*** (***h1, h2, …, hn***) as input data that has **n features,** perform dot product with 1 set of ***n*** weights (***w1, w2, …, wn***) to get your final output **y_hat**.

Example of A Multi-Layer Neural Network with 784 input neurons, 16 hidden neurons, and 10 output neurons.



Backward propagation (Backward pass/Backpropagation)



1. receive new observation $\boldsymbol{x} = [x_1 \ldots x_d]$ and target y^*
2. **feed forward:** for each unit g_j in each layer 1…L
   compute g_j based on units f_k from previous layer: $g_j = \sigma\left(u_{j0} + \sum_k u_{jk} f_k\right)$
3. get prediction y and error $(y\text{-}y^*)$
4. **back-propagate error:** for each unit g_j in each layer L…1

(a) compute error on g_j

$$\frac{\partial E}{\partial g_j} = \sum_i \sigma'(h_i) v_{ij} \frac{\partial E}{\partial h_i}$$

- $\frac{\partial E}{\partial g_j}$: should g_j be higher or lower?
- $\sigma'(h_i) v_{ij}$: how h_i will change as g_j changes
- $\frac{\partial E}{\partial h_i}$: was h_i too high or too low?

(b) for each u_{jk} that affects g_j

(i) compute error on u_{jk}

$$\frac{\partial E}{\partial u_{jk}} = \frac{\partial E}{\partial g_j} \sigma'(g_j) f_k$$

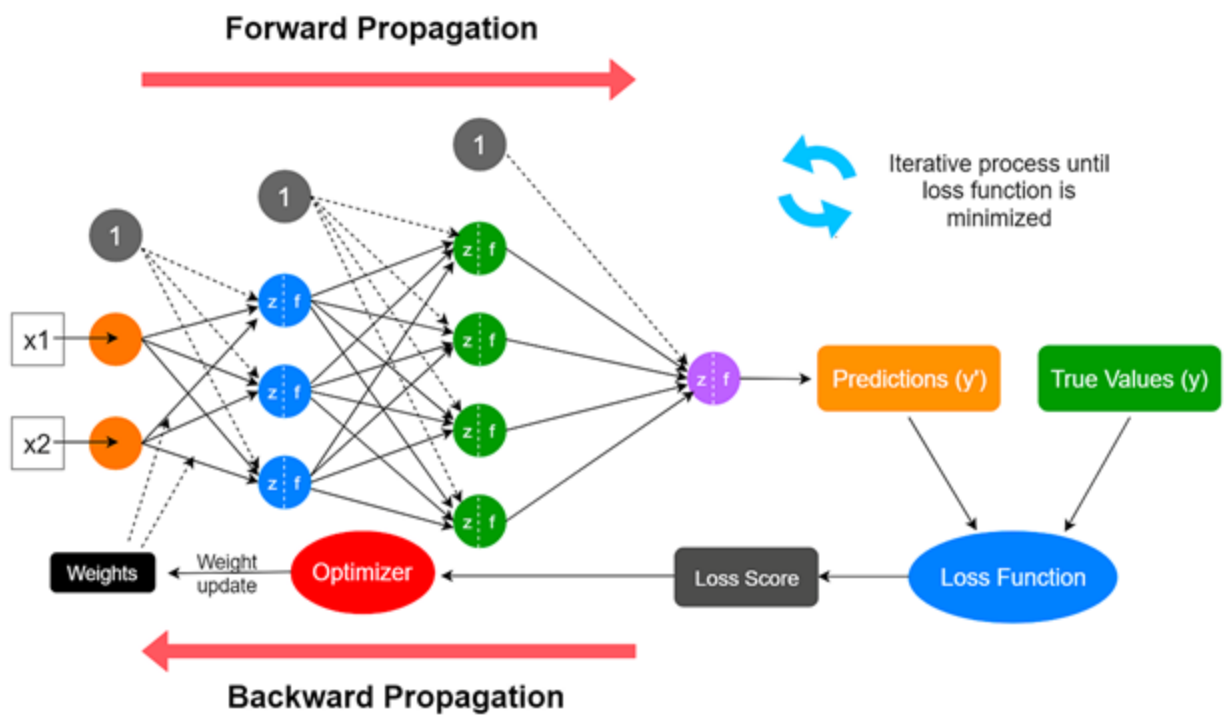
- $\frac{\partial E}{\partial g_j}$: do we want g_j to be higher/lower
- $\sigma'(g_j) f_k$: how g_j will change if u_{jk} is higher/lower

(ii) update the weight

$$u_{jk} \leftarrow u_{jk} - \eta \frac{\partial E}{\partial u_{jk}}$$

https://neptune.ai/blog/backpropagation-algorithm-in-neural-networks-guide

Calculation of the loss function



3 Key Loss Functions

- Mean Squared Error Loss Function
- Mean Absolute Percentage Error
- Cross-Entropy Loss Function

1. Mean Squared Error Loss Function

Mean squared error (MSE) loss function is the sum of squared differences between the entries in the prediction vector y and the ground truth vector y_hat.

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=0}^{N}(y_i - \hat{y}_i)^2$$

y_i : entries in the prediction vector $\vec{y}$

$\hat{y}_i$: entries in the ground truth label $\vec{\hat{y}}$

2

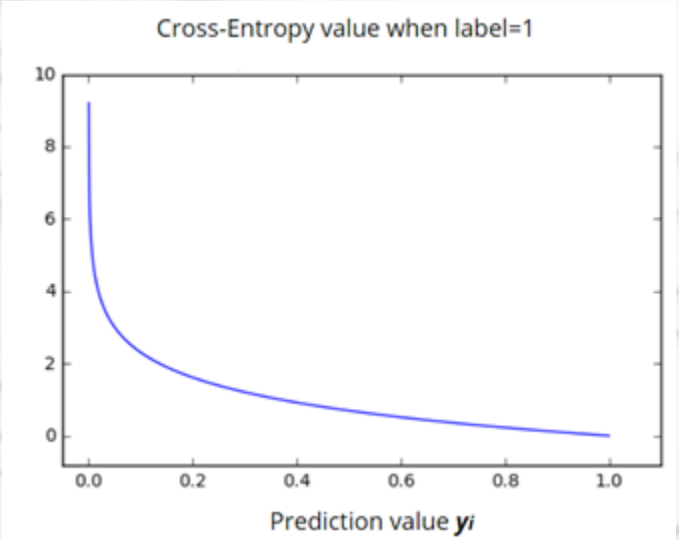
$$MAPE = \frac{100\%}{N}\sum_{i=0}^{N}\frac{|y_i - \hat{y}_i|}{\hat{y}_i}$$

3. Cross-Entropy Loss Function

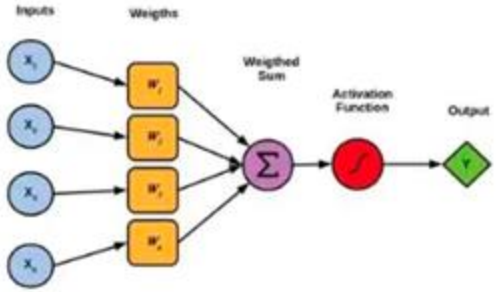
$$\mathcal{L}(\theta) = -\sum_{i=0}^{N}\hat{y}_i \cdot log(y_i)$$

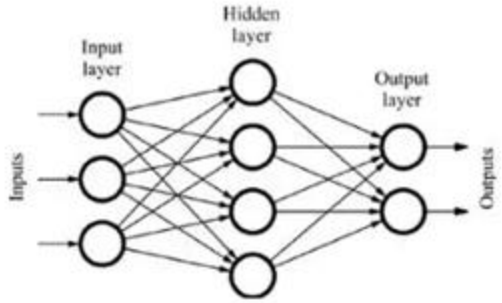
y_i : entries in the prediction vector $\vec{y}$

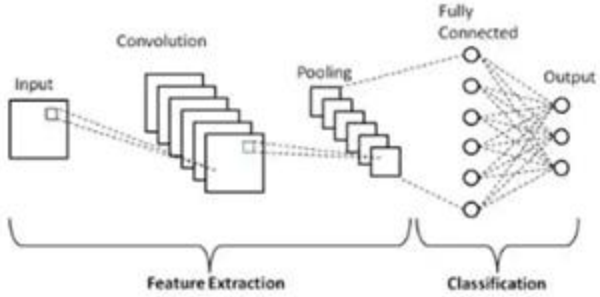
$\hat{y}_i$: entries in the ground truth label $\vec{\hat{y}}$

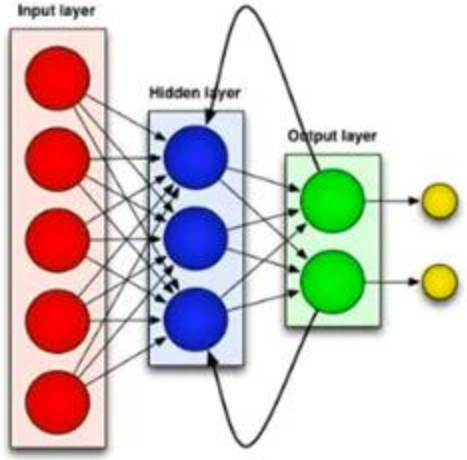


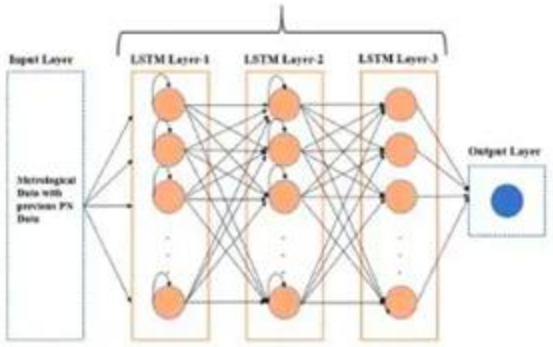
Types of neural networks

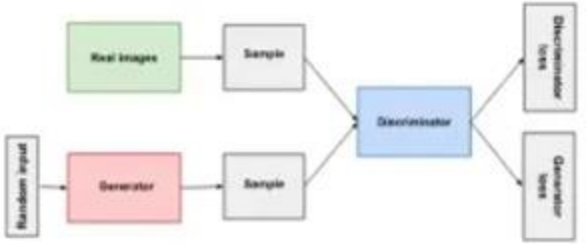












Perceptron

The **perceptron**, a ***foundational concept*** in artificial intelligence and machine learning, is a simplified model of a biological neuron. It's a simple algorithm used for binary classification tasks

A Brief History. The perceptron was introduced by Frank Rosenblatt in 1957. It was inspired by the workings of the human brain, where neurons process information through interconnected networks. Rosenblatt envisioned a machine that could learn and recognize patterns, much like a human brain.

Key Components of a Perceptron:

- Inputs
- Weights, Bias. A bias term is added to the weighted sum to adjust the decision boundary.
- Activation Function. Common activation functions include the step function, sigmoid function, and ReLU function.

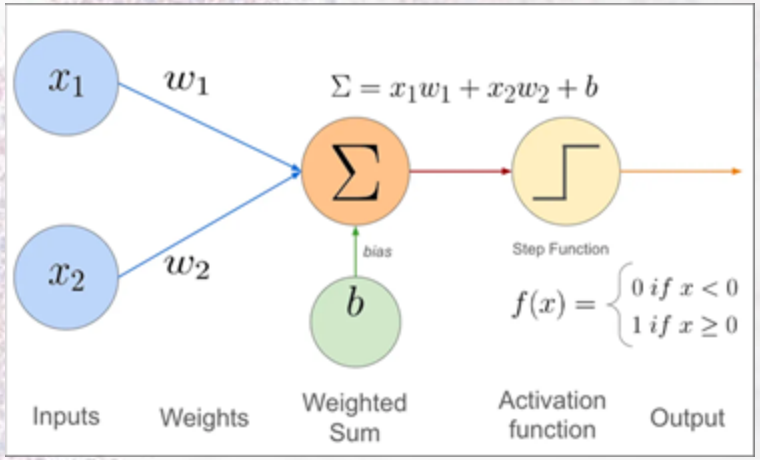
The Perceptron Learning Rule is an iterative algorithm used to adjust the weights and bias of the perceptron based on the error between the predicted output and the actual output. This process continues until the perceptron can accurately classify the training data.

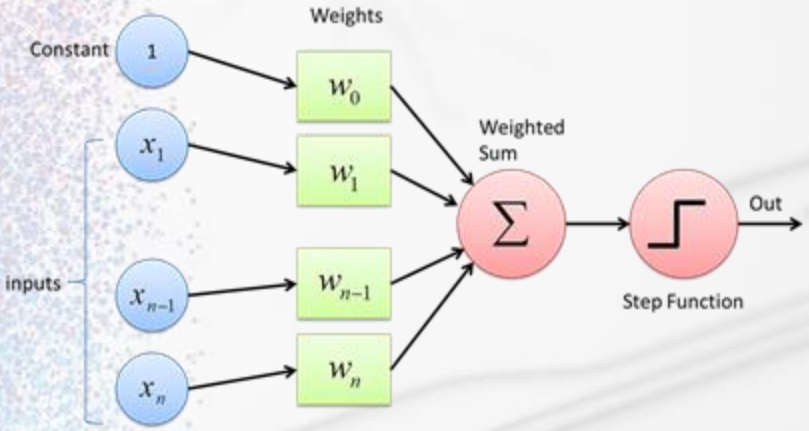
Limitations of the Perceptron. While the perceptron is a valuable tool for understanding neural networks, it has limitations. It can only classify linearly separable data, meaning it cannot handle complex patterns that require non-linear decision boundaries. This limitation led to a period known as the "AI winter" in the 1970s.

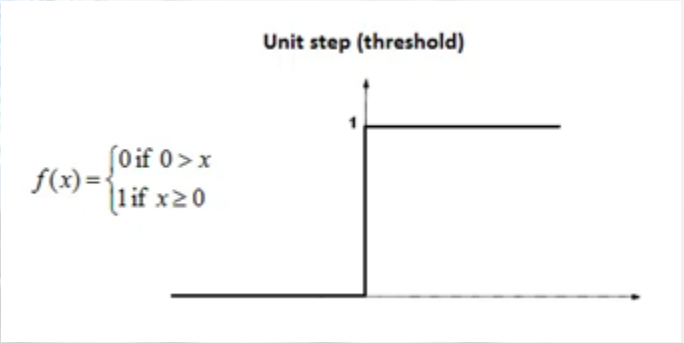
Ref:

→https://pub.aimind.so/perceptron-101-the-building-blocks-of-a-neural-network-496f6b9b3826

→https://www.youtube.com/watch?v=-KLnurhX-Pg







Perceptron

How a Perceptron Works

- **Input** receives multiple input signals, each representing a feature of the data.
- **Weights and Bias.** Each input is multiplied by a corresponding weight. These weights determine the importance of each input. A bias term is added to the weighted sum. This bias allows the model to adjust the decision boundary.
- **Weighted Sum.**

  *weighted_sum = (input1 * weight1) + (input2 * weight2) + ... + bias*
- **Activation Function.** The weighted sum is passed through an activation function, which introduces non-linearity. A common activation function is the step function:

  output = 1 if weighted_sum >= threshold else 0

  - If the weighted sum exceeds a certain threshold, the output is 1 (positive class).
  - Otherwise, the output is 0 (negative class).

Learning Process:

The perceptron learns by adjusting its weights and bias through a process called training. This involves:

1. **Initialization.** The weights and bias are initialized to random values.
2. **Forward Pass.** The input data is fed into the perceptron, and the output is calculated.
3. **Error Calculation.** The error between the predicted output and the actual output is calculated.
4. **Weight Update.** The weights and bias are updated using a learning rule, such as the perceptron learning rule, to minimize the error:

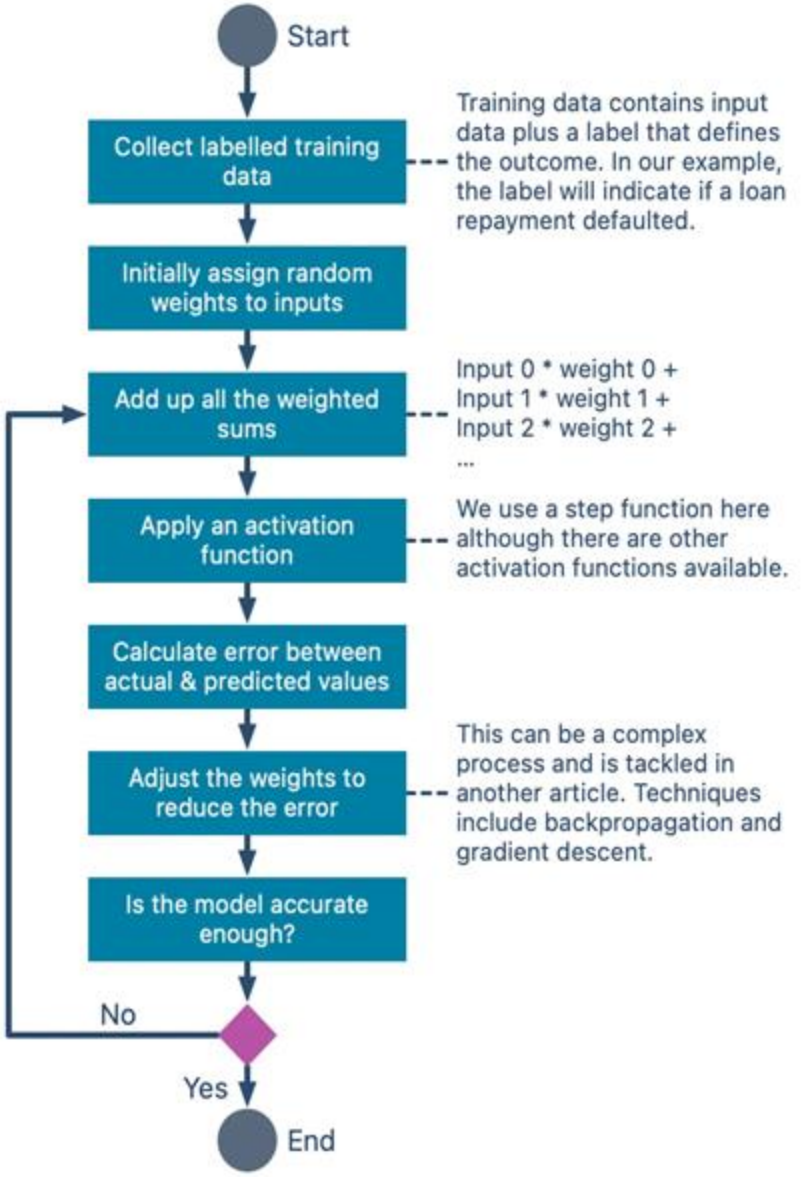
   *weight_new = weight_old + learning_rate * error * input*

   *bias_new = bias_old + learning_rate * error*

1. **Iterative Process** is repeated for multiple training examples, iteratively adjusting the weights and bias until the perceptron can accurately classify the data.

Example

→ https://www.kaggle.com/code/kuzagaya/implementing-a-perceptron-in-python



Radial Basis Function network - RBF

Radial Basis Function (RBF) networks are a type of artificial neural network that uses radial basis functions as activation functions. They are particularly well-suited for function approximation, time series prediction, and classification tasks.

Three-Layer Architecture

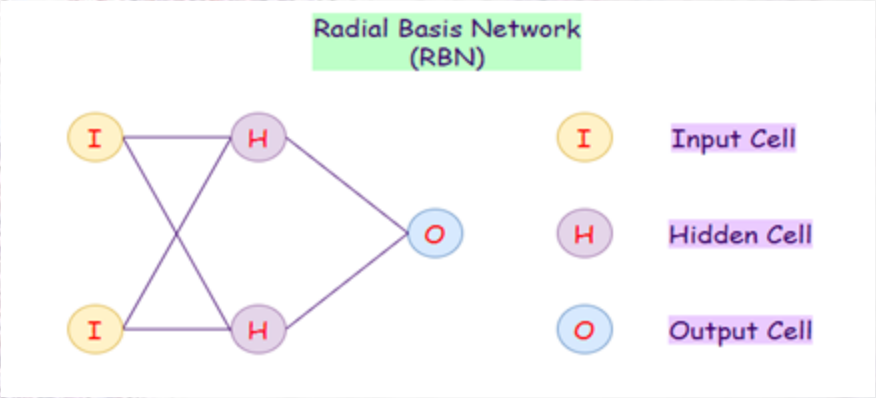
- Input Layer receives input data and passes it to the hidden layer.
- Hidden Layer composed of RBF neurons, each with its own center and width. The activation of each neuron depends on the distance between the input and its center.
- Output Layer performs a linear combination of the hidden layer outputs to produce the final output.

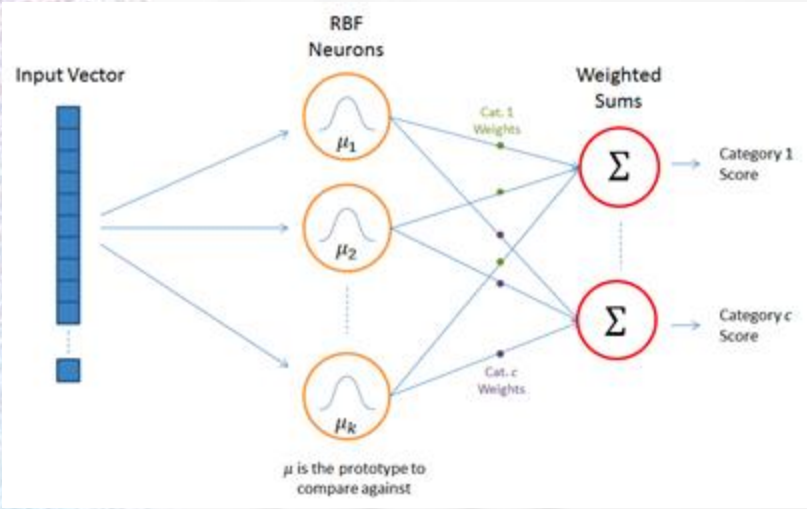
Radial Basis Functions

- These functions are characterized by their radial symmetry, meaning their output depends solely on the distance from a central point.
- Common RBFs include:
  - Gaussian RBF
  - Multiquadric RBF
  - Inverse Multiquadric RBF
  - Thin Plate Spline RBF

Training Process

- Center Determination. The centers of the RBF neurons can be determined using various methods, such as clustering algorithms (e.g., K-means) or random selection.
- Width Determination. The widths of the RBF neurons control their influence on the output. They can be fixed or learned during training.
- Weight Training. The weights of the connections between the hidden and output layers are typically trained using linear least squares methods.



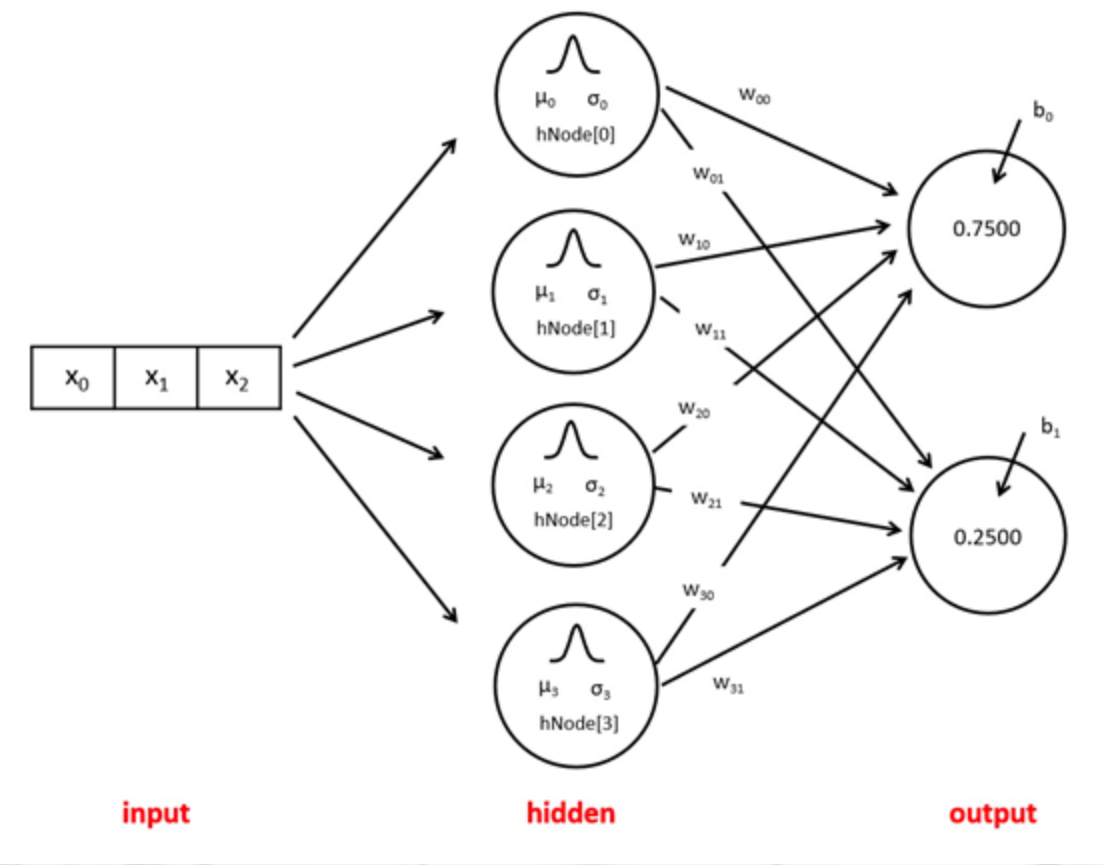


$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Radial Basis Function network - RBF

Radial Basis Networks and Custom Keras Layers
→https://kaggle.com/code/residentmario/radial-basis-networks-and-custom-keras-layers

Radial Basis Interpolation From Scratch Using Python
→https://www.youtube.com/watch?v=Kl9xk9BukNE



Advantages of RBF Networks:

- **Universal Approximation:** RBF networks can approximate any continuous function to arbitrary accuracy.
- **Faster Training:** Compared to other neural networks, RBF networks often require less training time.
- **Good Generalization Performance:** RBF networks tend to generalize well to unseen data.

Disadvantages of RBF Networks:

- **Sensitivity to Center and Width:** The performance of RBF networks is sensitive to the choice of centers and widths.
- **Potential Overfitting:** RBF networks with too many hidden neurons can overfit the training data.

Applications of RBF Networks:

- **Function Approximation:** Modeling complex functions, such as nonlinear systems.
- **Time Series Prediction:** Forecasting future values based on past data.
- **Classification:** Distinguishing between different categories of data.
- **Control Systems:** Designing controllers for dynamic systems.
- **Pattern Recognition:** Identifying patterns in data, such as image and speech recognition.

Multilayer Perceptron (MLP)

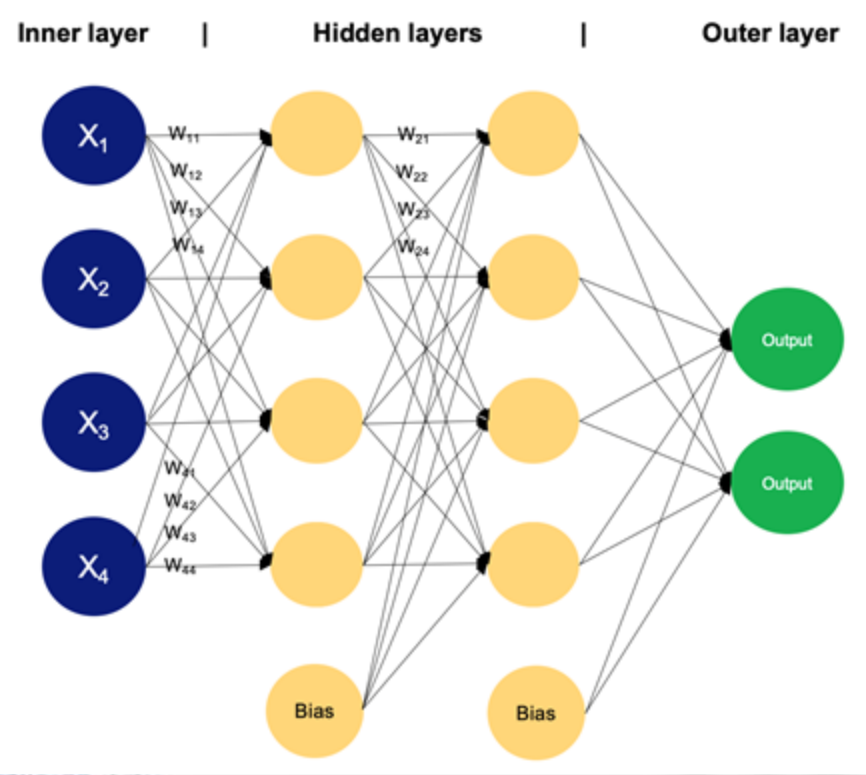
MLPs are a specific type of feedforward neural network that consists of multiple layers of interconnected neurons. Each neuron in a layer is connected to every neuron in the next layer. The multiple layers allow the network to extract features from the input data and gradually build up a representation of the underlying patterns.

Key Components of an MLP:

1. **Input Layer r**eceives input data and passes it to the hidden layer(s).
2. **Hidden Layer(s)** process the input data and pass the results to the output layer. These layers introduce non-linearity to the network, enabling it to learn complex patterns.
3. **Output Layer** produces the final output of the network.

Activation Functions. Some common activation functions used in MLPs include:

- **Sigmoid:** Squashes the input to a value between 0 and 1.
- **Tanh:** Similar to sigmoid, but outputs values between -1 and 1.
- **ReLU (Rectified Linear Unit):** Outputs the maximum of 0 and the input.



Sigmoid	Tanh	ReLU	Leaky ReLU
$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$

$$\text{Weighted Sum} = \sum_{i=1}^{n} (w_i * x_i) + b$$

Multilayer Perceptron (MLP)

Training Algorithms for MLPs

One of the most crucial aspects of MLPs is the training process, which involves adjusting the weights and biases of the network to minimize the error between the predicted output and the actual output.

The most widely used algorithm for training MLPs is backpropagation.

Backpropagation

1. **Forward Pass:**
- Input data is fed into the network.
- The data propagates through the layers, with each neuron computing its activation function.
- The output layer produces a prediction.

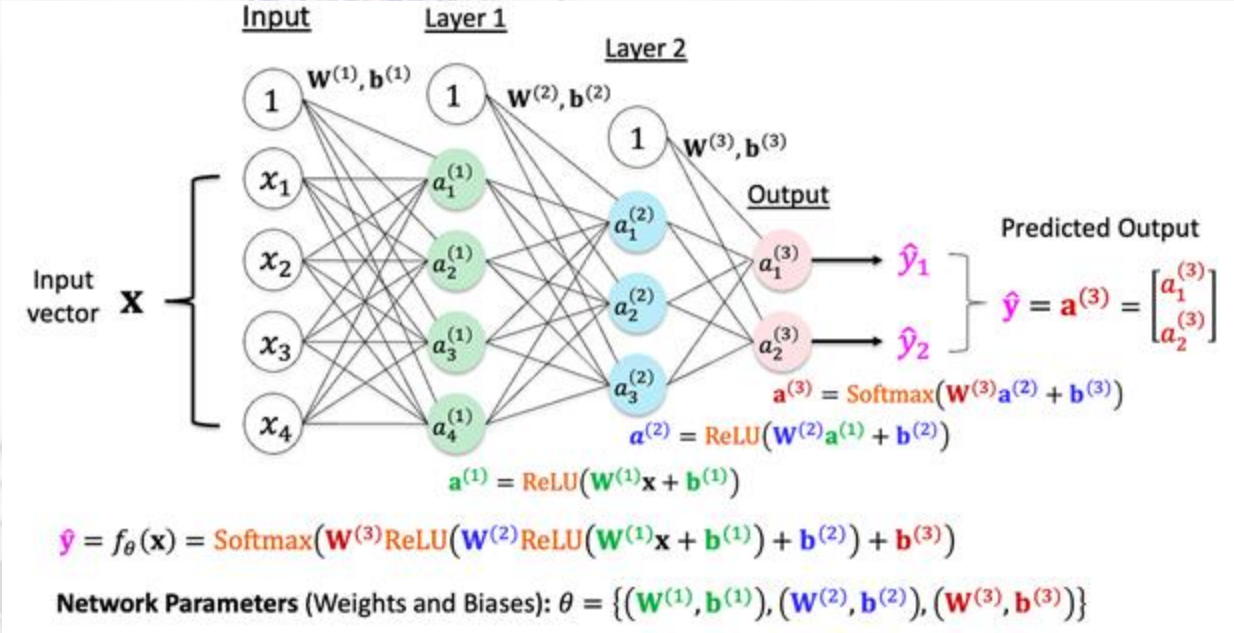
1. **Backward Pass:**
- The error between the predicted output and the actual output is calculated.
- This error is propagated backward through the network, layer by layer.
- The weights and biases of each neuron are adjusted using gradient descent to minimize the error.

MLP example

→https://github.com/rcassani/mlp-example/blob/master/mlp_examples.ipynb

Regularization Techniques. To mitigate overfitting, regularization techniques can be employed:

- **L1 Regularization:** Adds the absolute values of the weights to the loss function.
- **L2 Regularization (Weight Decay):** Adds the squared values of the weights to the loss function.
- **Dropout:** Randomly drops out neurons during training to prevent co-adaptation.



References

1) Artificial Intelligence Full Course | Artificial Intelligence Tutorial for Beginners | Edureka
https://www.youtube.com/watch?v=JMUxmLyrhSk

2) Full Machine learning Tutorial
https://www.youtube.com/playlist?list=PLEiEAq2VkUUKiW6VQ0iMjsCpjN2rqG_RO

3) **But what is a neural network?**
→ https://www.youtube.com/watch?v=aircAruvnKk

4) **Gradient descent, how neural networks learn**
→ https://www.youtube.com/watch?v=IHZwWFHWa-w

5) How Computer Vision Applications Work
https://www.youtube.com/watch?v=oGvHtpJMO3M

6) 25 AI Concepts EVERYONE Should Know
https://www.youtube.com/watch?v=0uSyM2DZoOU

7) **Why Neural Networks can learn (almost) anything**

https://www.youtube.com/watch?v=0QczhVg5HaI